

PROGRAM DSA/小班课

[DSA 课程网站](#)

常欣海

PROGAM 教学计划

Course Syllabus

数算 B 小班课教学计划

周次	日期	时间	主题
3	3 月 21 日周六	18:40-20:30	基础、复杂度与线性表
6	4 月 11 日周六	18:40-20:30	链表、串与线性结构应用
8	4 月 25 日周六	18:40-20:30	排序、树与检索
10	5 月 9 日周六	18:40-20:30	图算法与综合收束
13	5 月 30 日周六	18:40-20:30	期末上机复习
14	6 月 6 日周六	18:40-20:30	期末笔试复习

PROGRAM

L2 链表、 串与线性结构应用

常欣海

2026.04.11

PROGRAM 算法

算法定义

Algorithm

算法 (algorithm) 是在有限时间内解决特定问题的一组指令或操作步骤，它具有以下特性。

- 问题是明确的，包含清晰的输入和输出定义。
- 具有可行性，能够在有限步骤、时间和内存空间下完成。
- 各步骤都有确定的含义，在相同的输入和运行条件下，输出始终相同。



扑克排序

插入排序算法



货币找零

贪心算法

顺序表

Array

定义

数组 (array)

是一种线性数据结构，其将相同类型的元素存储在连续的内存空间中。我们将元素在数组中的位置称为该元素的索引 (index)



存储数组的内存空间是连续的



可用内存空间



划分给数组的内存空间

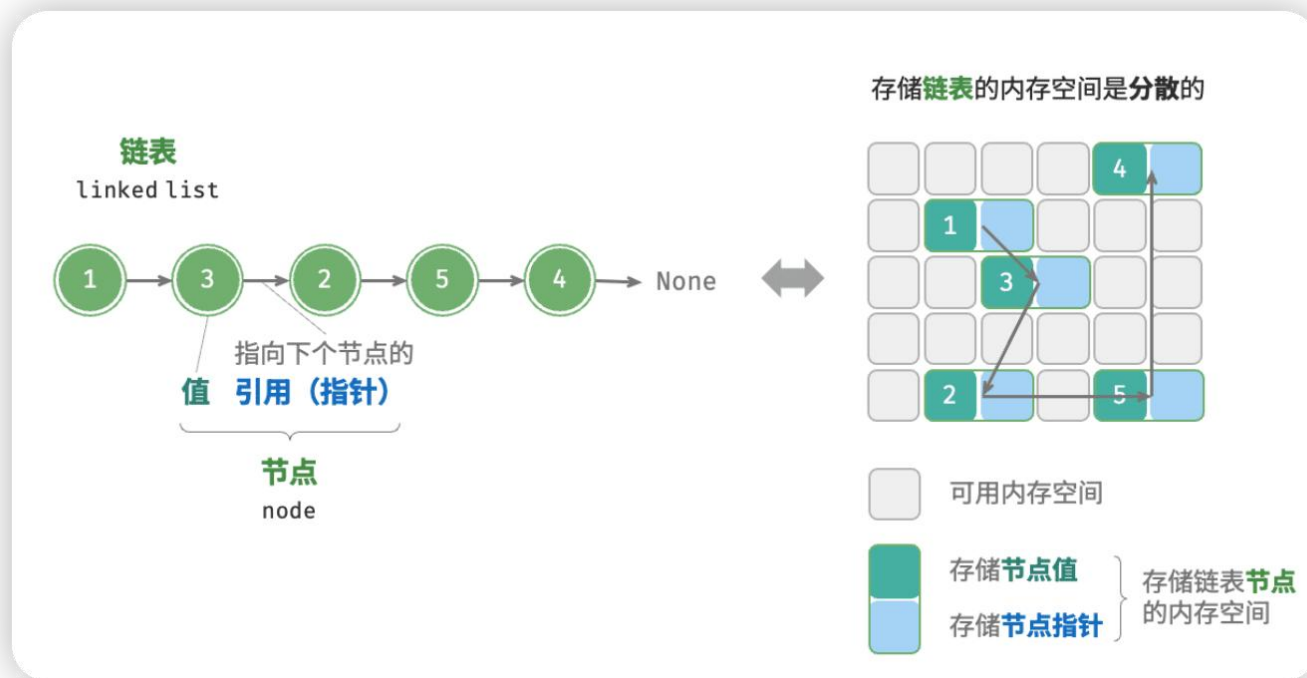
链表

Array

定义

链表 (linked list)

是一种线性数据结构，其中的每个元素都是一个节点对象，各个节点通过“引用”相连接。引用记录了下一个节点的内存地址，通过它可以从当前节点访问到下一个节点。



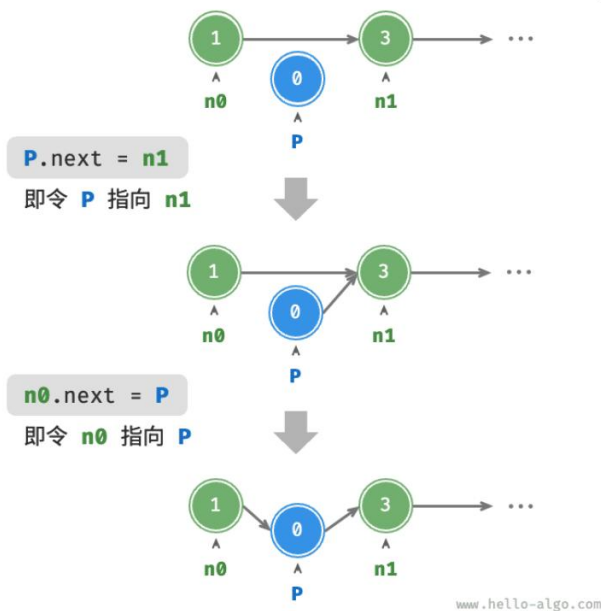
链表

Array

常用操作

示例

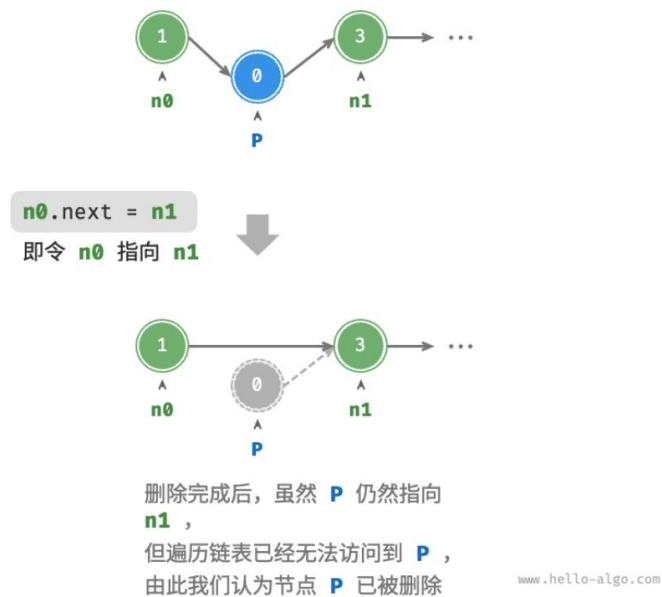
在链表节点 $n0$ 和 $n1$ 之间插入节点 P



插入节点

示例

在链表中删除节点 P



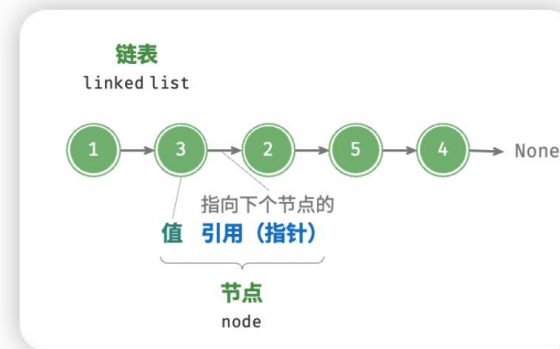
删除节点

代码

链表

Array

```
class ListNode:
    """链表节点类"""
    def __init__(self, val: int):
        self.val: int = val # 节点值
        self.next: ListNode | None = None # 指向下一节点的引用
```



```
# 初始化链表 1 -> 3 -> 2 -> 5 -> 4
# 初始化各个节点
n0 = ListNode(1)
n1 = ListNode(3)
n2 = ListNode(2)
n3 = ListNode(5)
n4 = ListNode(4)
# 构建节点之间的引用
n0.next = n1
n1.next = n2
n2.next = n3
n3.next = n4
```

```
def insert(n0: ListNode, P: ListNode):
    """在链表的节点 n0 之后插入节点 P"""
    n1 = n0.next
    P.next = n1
    n0.next = P
```

```
def remove(n0: ListNode):
    """删除链表的节点 n0 之后的首个节点"""
    if not n0.next:
        return
    # n0 -> P -> n1
    P = n0.next
    n1 = P.next
    n0.next = n1
```

```
def access(head: ListNode, index: int) -> ListNode | None:
    """访问链表中索引为 index 的节点"""
    for _ in range(index):
        if not head:
            return None
        head = head.next
    return head
```

```
def find(head: ListNode, target: int) -> int:
    """在链表中查找值为 target 的首个节点"""
    index = 0
    while head:
        if head.val == target:
            return index
        head = head.next
        index += 1
    return -1
```

PROGRAM

栈

Stack

定义

栈（Stack）一种遵循先入后出逻辑的线性数据结构

LIFO

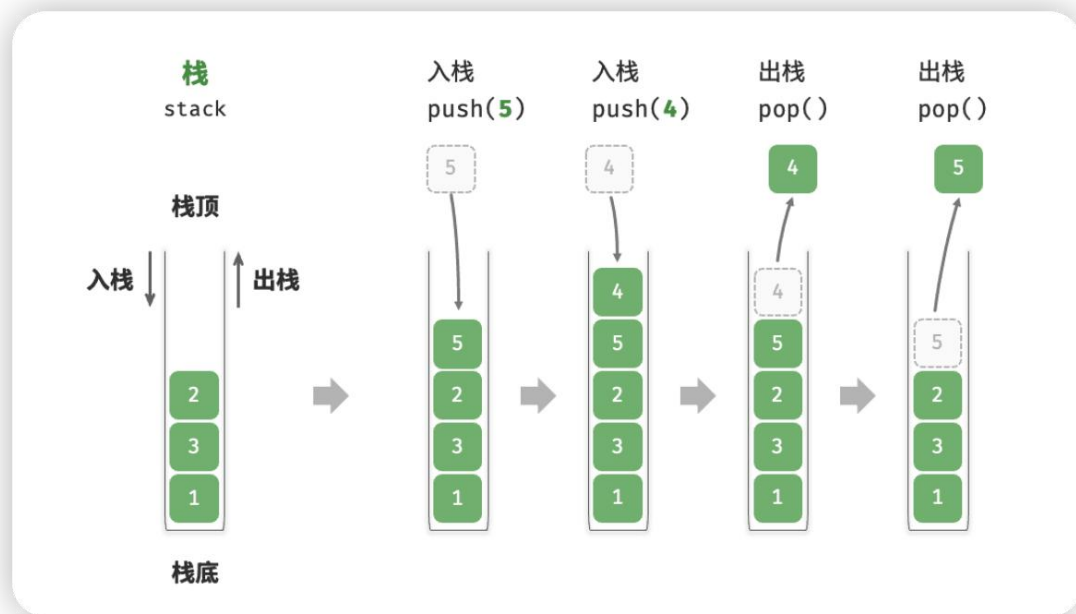


表 5-1 栈的操作效率

方法	描述	时间复杂度
push()	元素入栈（添加至栈顶）	$O(1)$
pop()	栈顶元素出栈	$O(1)$
peek()	访问栈顶元素	$O(1)$

PROGRAM 栈

代码

算法实现

```
# 初始化栈
# Python 没有内置的栈类，可以把 list 当作栈来使用
stack: list[int] = []

# 元素入栈
stack.append(1)
stack.append(3)
stack.append(2)
stack.append(5)
stack.append(4)

# 访问栈顶元素
peek: int = stack[-1]

# 元素出栈
pop: int = stack.pop()

# 获取栈的长度
size: int = len(stack)

# 判断是否为空
is_empty: bool = len(stack) == 0
```

列表实现

Stack

算法原理

```
class LinkedListStack:
    """基于链表实现的栈"""

    def __init__(self):
        """构造方法"""
        self._peek: ListNode | None = None
        self._size: int = 0

    def size(self) -> int:
        """获取栈的长度"""
        return self._size

    def is_empty(self) -> bool:
        """判断栈是否为空"""
        return self._size == 0

    def push(self, val: int):
        """入栈"""
        node = ListNode(val)
        node.next = self._peek
        self._peek = node
        self._size += 1
```

链表实现

```
class ArrayStack:
    """基于数组实现的栈"""

    def __init__(self):
        """构造方法"""
        self._stack: list[int] = []

    def size(self) -> int:
        """获取栈的长度"""
        return len(self._stack)

    def is_empty(self) -> bool:
        """判断栈是否为空"""
        return self.size() == 0

    def push(self, item: int):
        """入栈"""
        self._stack.append(item)

    def pop(self) -> int:
        """出栈"""
        if self.is_empty():
            raise IndexError("栈为空")
        return self._stack.pop()
```

数组实现

PROGRAM

栈

Stack

软件中的撤销与反撤销

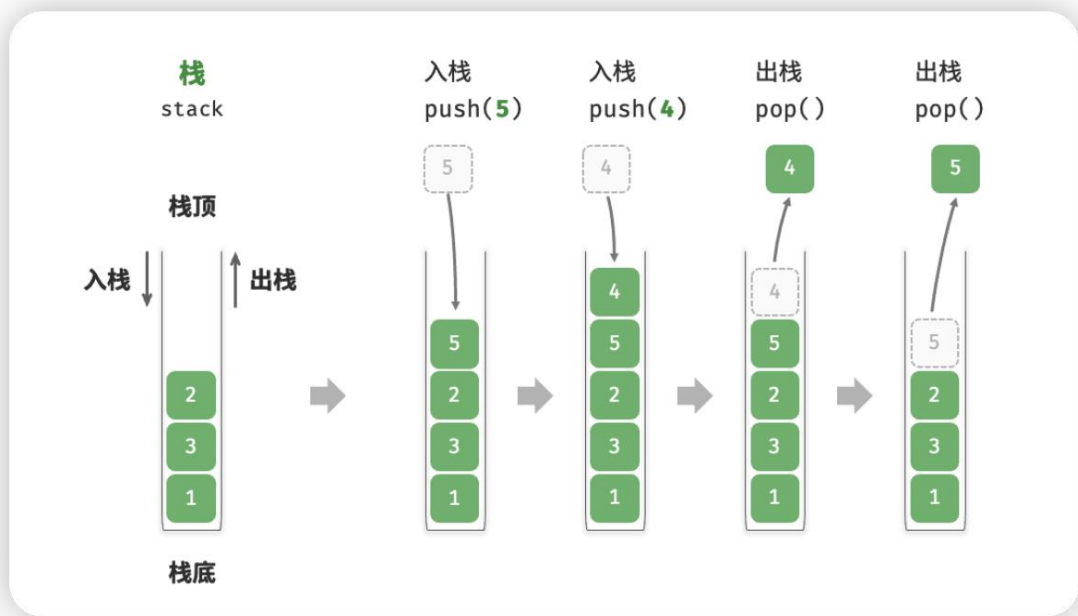
定义

栈（Stack）一种遵循先入后出逻辑的线性数据结构

LIFO

表 5-1 栈的操作效率

方法	描述	时间复杂度
push()	元素入栈（添加至栈顶）	$O(1)$
pop()	栈顶元素出栈	$O(1)$
peek()	访问栈顶元素	$O(1)$



PROGRAM

栈

Stack

代码

e.g. 用两个栈协同实现双向历史导航

操作	back_stack	forward_stack	current
初始 home	[]	[]	home
visit page1	[home]	[]	page1
visit page2	[home, page1]	[]	page2
visit page3	[home, page1, page2]	[]	page3
back	[home, page1]	[page3]	page2
back	[home]	[page3, page2]	page1
forward	[home, page1]	[page3]	page2
visit page4	[home, page1, page2]	[]	page4 ← 清空前进历史

```
class BrowserHistory:
    """双栈实现浏览器后退与前进"""

    def __init__(self, homepage: str):
        self.back_stack: list[str] = [] # 后退栈
        self.forward_stack: list[str] = [] # 前进栈
        self.current = homepage

    def visit(self, url: str) -> None:
        """访问新页面: 当前页入后退栈, 清空前进栈"""
        self.back_stack.append(self.current)
        self.current = url
        self.forward_stack.clear() # 新访问路径清除前进历史

    def back(self) -> str | None:
        """后退: 当前页入前进栈, 后退栈栈顶弹出设为当前"""
        if not self.back_stack:
            return None
        self.forward_stack.append(self.current)
        self.current = self.back_stack.pop()
        return self.current

    def forward(self) -> str | None:
        """前进: 当前页入后退栈, 前进栈栈顶弹出设为当前"""
        if not self.forward_stack:
            return None
        self.back_stack.append(self.current)
        self.current = self.forward_stack.pop()
        return self.current
```


PROGRAM 队列

定义

Queue

代办事项

队列（queue）是一种遵循先入先出规则的线性数据结构 **FIFO**

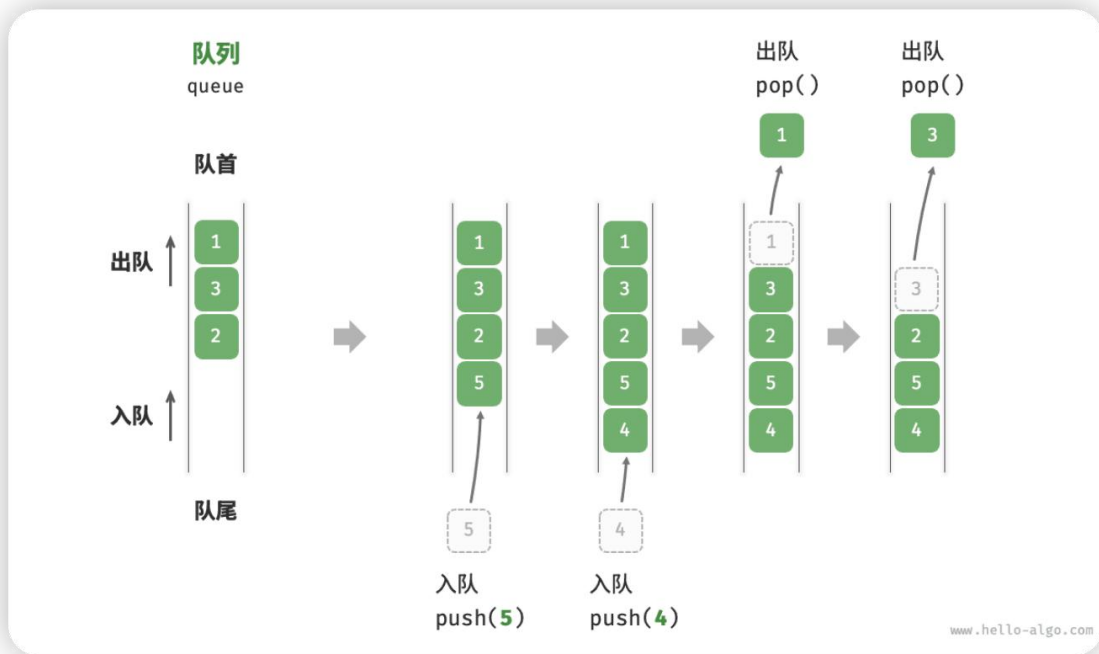


表 5-2 队列操作效率

方法名	描述	时间复杂度
push()	元素入队，即将元素添加至队尾	$O(1)$
pop()	队首元素出队	$O(1)$
peek()	访问队首元素	$O(1)$

PROGRAM 队列

代码

算法实现

Queue

算法原理

```
from collections import deque

# 初始化队列
# 在 Python 中，我们一般将双向队列类 deque 当作队列使用
# 虽然 queue.Queue() 是纯正的队列类，但不好用，因此不推荐
que: deque[int] = deque()

# 元素入队
que.append(1)
que.append(3)
que.append(2)
que.append(5)
que.append(4)

# 访问队首元素
front: int = que[0]

# 元素出队
pop: int = que.popleft()

# 获取队列的长度
size: int = len(que)

# 判断队列是否为空
is_empty: bool = len(que) == 0
```

列表实现

```
class LinkedListQueue:
    """基于链表实现的队列"""

    def __init__(self):
        """构造方法"""
        self._front: ListNode | None = None # 头节点 front
        self._rear: ListNode | None = None # 尾节点 rear
        self._size: int = 0

    def size(self) -> int:
        """获取队列的长度"""
        return self._size

    def is_empty(self) -> bool:
        """判断队列是否为空"""
        return self._size == 0

    def push(self, num: int):
        """入队"""
        # 在尾节点后添加 num
        node = ListNode(num)
        # 如果队列为空，则令头、尾节点都指向该节点
        if self._front is None:
            self._front = node
            self._rear = node
        # 如果队列不为空，则将该节点添加到尾节点后
```

链表实现

```
class ArrayQueue:
    """基于环形数组实现的队列"""

    def __init__(self, size: int):
        """构造方法"""
        self._nums: list[int] = [0] * size # 用于存储队列元素的数组
        self._front: int = 0 # 队首指针，指向队首元素
        self._size: int = 0 # 队列长度

    def capacity(self) -> int:
        """获取队列的容量"""
        return len(self._nums)

    def size(self) -> int:
        """获取队列的长度"""
        return self._size

    def is_empty(self) -> bool:
        """判断队列是否为空"""
        return self._size == 0

    def push(self, num: int):
        """入队"""
        if self._size == self.capacity():
            raise IndexError("队列已满")
        # 计算队尾指针，指向队尾索引 + 1
        # 通过取余操作实现 rear 越过数组尾部后回到头部
        rear: int = (self._front + self._size) % self.capacity()
        # 将 num 添加至队尾
```

数组实现

双向队列

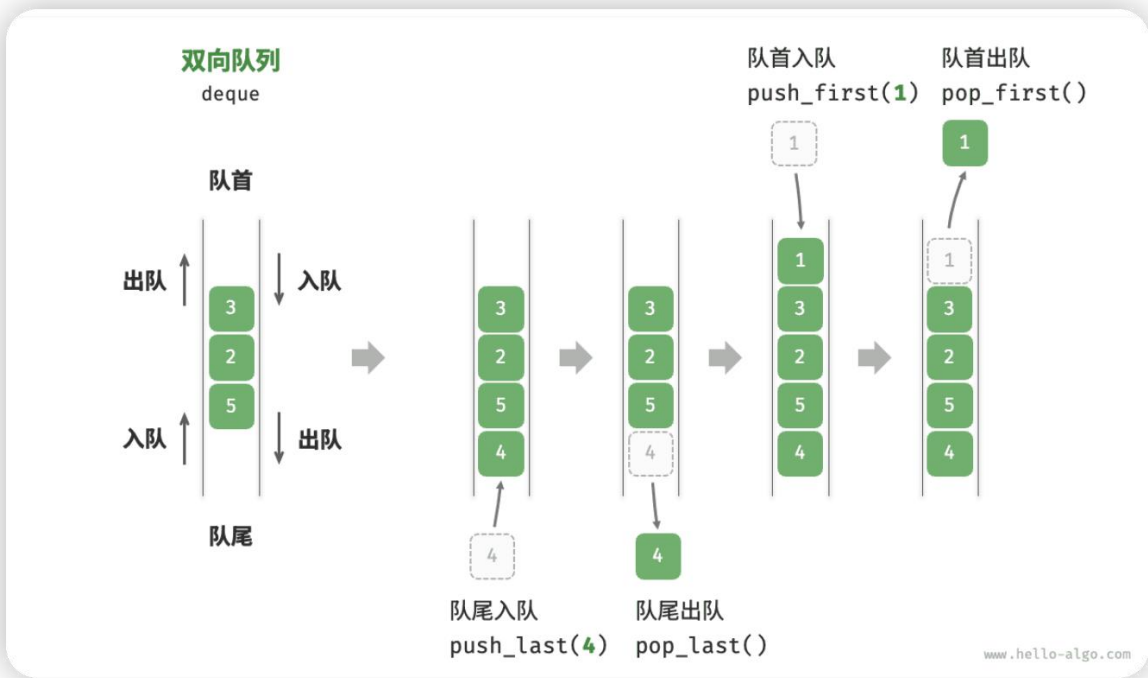
定义

Double-ended Queue

双向队列 允许在头部和尾部执行元素的添加或删除操作。

表 5-3 双向队列操作效率

方法名	描述	时间复杂度
<code>push_first()</code>	将元素添加至队首	$O(1)$
<code>push_last()</code>	将元素添加至队尾	$O(1)$
<code>pop_first()</code>	删除队首元素	$O(1)$
<code>pop_last()</code>	删除队尾元素	$O(1)$
<code>peek_first()</code>	访问队首元素	$O(1)$
<code>peek_last()</code>	访问队尾元素	$O(1)$



双向队列

Double-ended Queue

代码

算法实现

算法原理

```
from collections import deque

# 初始化双向队列
deq: deque[int] = deque()

# 元素入队
deq.append(2)      # 添加至队尾
deq.append(5)
deq.append(4)
deq.appendleft(3)  # 添加至队首
deq.appendleft(1)

# 访问元素
front: int = deq[0] # 队首元素
rear: int = deq[-1] # 队尾元素

# 元素出队
pop_front: int = deq.popleft() # 队首元素出队
pop_rear: int = deq.pop()      # 队尾元素出队

# 获取双向队列的长度
size: int = len(deq)

# 判断双向队列是否为空
is_empty: bool = len(deq) == 0
```

列表实现

```
class ListNode:
    """双向链表节点"""

    def __init__(self, val: int):
        """构造方法"""
        self.val: int = val
        self.next: ListNode | None = None # 后继节点引用
        self.prev: ListNode | None = None # 前驱节点引用

class LinkedListDeque:
    """基于双向链表实现的双向队列"""

    def __init__(self):
        """构造方法"""
        self._front: ListNode | None = None # 头节点 front
        self._rear: ListNode | None = None # 尾节点 rear
        self._size: int = 0 # 双向队列的长度

    def size(self) -> int:
        """获取双向队列的长度"""
        return self._size

    def is_empty(self) -> bool:
        """判断双向队列是否为空"""
        return self._size == 0
```

链表实现

```
class ArrayDeque:
    """基于环形数组实现的双向队列"""

    def __init__(self, capacity: int):
        """构造方法"""
        self._nums: list[int] = [0] * capacity
        self._front: int = 0
        self._size: int = 0

    def capacity(self) -> int:
        """获取双向队列的容量"""
        return len(self._nums)

    def size(self) -> int:
        """获取双向队列的长度"""
        return self._size

    def is_empty(self) -> bool:
        """判断双向队列是否为空"""
        return self._size == 0

    def index(self, i: int) -> int:
        """计算环形数组索引"""
        # 通过取余操作实现数组首尾相连
        # 当 i 越过数组尾部后, 回到头部
        # 当 i 越过数组头部后, 回到尾部
        return (i + self.capacity()) % self.capacity()
```

数组实现

定义

双向队列

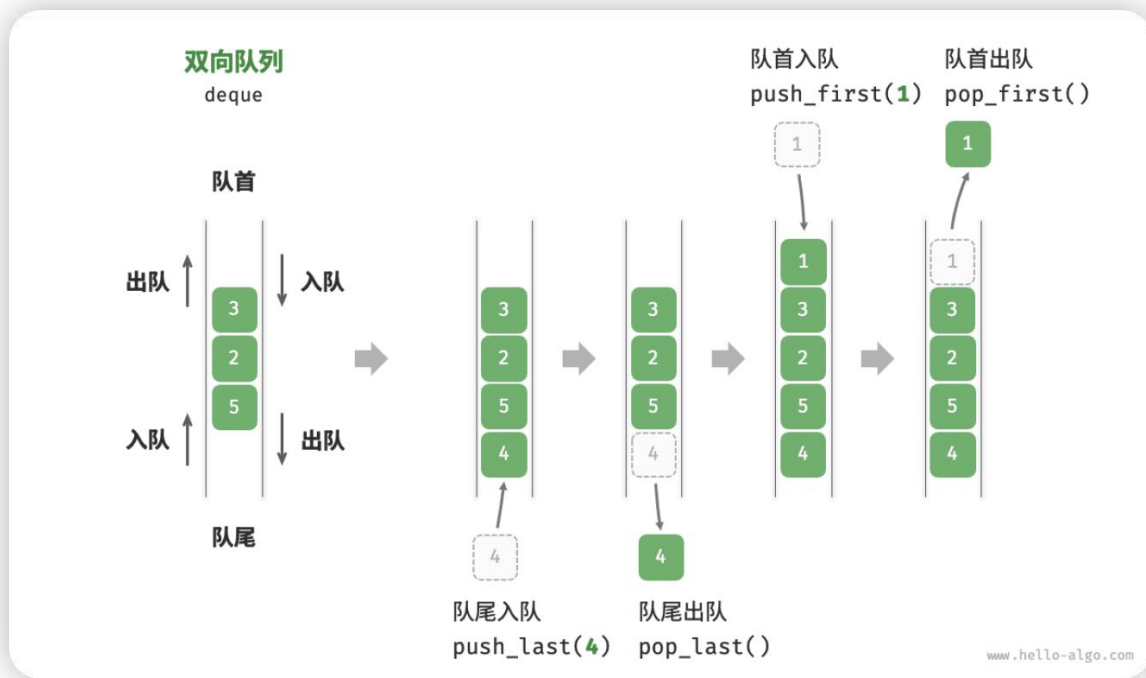
Double-ended Queue

软件的“撤销”功能
(无限-栈; 有限-双向队列)

双向队列 允许在头部和尾部执行元素的添加或删除操作。

表 5-3 双向队列操作效率

方法名	描述	时间复杂度
<code>push_first()</code>	将元素添加至队首	$O(1)$
<code>push_last()</code>	将元素添加至队尾	$O(1)$
<code>pop_first()</code>	删除队首元素	$O(1)$
<code>pop_last()</code>	删除队尾元素	$O(1)$
<code>peek_first()</code>	访问队首元素	$O(1)$
<code>peek_last()</code>	访问队尾元素	$O(1)$



双向队列

代码

Double-ended Queue

操作	动作 / 撤销	deque (队首 → 队尾)	current
—	初始状态	[]	
do	写代码 A	[A]	A
do	写代码 B	[A, B]	B
do	写代码 C	[A, B, C]	C
do	写代码 D	[A, B, C, D]	D
△do	写代码 E	[B, C, D, E] (A被丢弃)	E
undo	撤销「E」	[B, C, D]	D
undo	撤销「D」	[B, C]	C
undo	撤销「C」	[B]	B
undo	撤销「B」	[]	
undo	撤销失败	[]	

```
from collections import deque

class UndoManager:
    """基于双向队列的有限撤销功能"""

    def __init__(self, max_steps: int = 5):
        self.max_steps = max_steps
        self.undo_deque: deque[str] = deque(maxlen=max_steps)
        self.current: str = "" # 当前状态

    def do(self, action: str) -> str:
        """执行操作 (入队), 返回操作描述"""
        self.undo_deque.append(action)
        self.current = action
        return action

    def undo(self) -> str | None:
        """撤销操作 (队尾出), 返回被撤销的操作"""
        if not self.undo_deque:
            return None
        action = self.undo_deque.pop()
        self.current = self.undo_deque[-1] if self.undo_deque else ""
        return action

    def get_deque_state(self) -> list[str]:
        """返回队列状态 (队首→队尾)"""
        return list(self.undo_deque)

    def get_current(self) -> str:
        """返回当前状态"""
        return self.current
```

PROGRAM 习题

Practice

洛谷 / 题单列表 / 题单详情

L2 Practice

收藏题单

题目总数3 | 收藏人数0

题单简介 | 题目列表

状态	题号	题目名称	显示来源	标签	难度	通过率
—	P1996	约瑟夫问题	模拟 队列	树状数组 链表	普及-	
—	P1449	后缀表达式	模拟 线性数据结构	字符串 栈	普及-	
—	P1160	队列安排	模拟 构造	线性数据结构 链表	普及/提高-	

多选

L2 Practice

PROGRAM

Q&A

Q&A